



Relatório do Trabalho Final
Introdução a Machine Learning

**Otimização de Farming em RPGs com
Aprendizado de Máquina: Um Sistema de
Recomendação para Maximização no Ganho
De Experiência em Pokémon**

Autores:

Giovanna Oliveira da Silva
Kayo Bacelar de Oliveira
Rodrigo da Silva Nogueira
Leonardo Ambrus de Lima
Isabella Vitoria da Silva Raymundo

27 de junho de 2026

Resumo

Este trabalho apresenta o desenvolvimento de um pipeline completo de Aprendizado de Máquina voltado à modelagem preditiva de combates e à formulação de um sistema de recomendação focado na otimização do ganho de pontos de Experiência (XP). Tomando como base o ecossistema competitivo da franquia Pokémon, o projeto utiliza algoritmos de classificação supervisionada para identificar oponentes que combinem vulnerabilidade matemática e alta rentabilidade de treino. A base de dados foi gerada a partir de um motor de batalhas determinístico em nível máximo (Level 100), consolidando um dataset sintético contendo 10.000 confrontos. O pipeline abrange desde o pré-processamento estruturado e engenharia de atributos — transformando relações elementares de tipos em features contínuas — até o ajuste fino de hiperparâmetros por validação cruzada exaustiva via *GridSearchCV*. Três arquiteturas preditivas foram avaliadas e contrastadas: Regressão Logística, *Random Forest* e *Multi-Layer Perceptron* (MLP). Como produto final, implementou-se um algoritmo recomendador para "*Farming Seguro*", capaz de filtrar adversários e ranqueá-los com base em um limiar dinâmico de probabilidade de vitória customizado pelo utilizador (configurável a partir de 50%), articulando conceitos práticos de viés algorítmico, eficiência e capacidade de generalização.

Palavras-chave: Aprendizado de Máquina, Sistema de Recomendação, Regressão Logística, Pokémon, Otimização de Processos.

1 Introdução

Nos jogos de interpretação de papéis (*Role-Playing Games* - RPGs), o progresso de um personagem está intrinsecamente atrelado à aquisição contínua de pontos de Experiência (XP). Esse processo de acumulação, popularmente denominado "*farming*" ou "*grinding*", frequentemente exige que o jogador enfrente uma sequência repetitiva de combates para elevar os atributos de suas unidades. Na franquia Pokémon, esse cenário ganha contornos complexos devido ao vasto arranjo de variáveis envolvidas nas dinâmicas de combate, tais como vantagens elementares de tipos, distribuição de estatísticas base, golpes disponíveis e níveis dos oponentes. Maximizar a eficiência dessa rotina consiste em encontrar oponentes que cedam grandes quantidades de experiência, mas que minimizem o risco de derrota ou o desgaste excessivo da equipe do jogador.

Diante desse cenário, este trabalho propõe a aplicação de técnicas de Aprendizado de Máquina Supervisionado para mapear e mitigar os riscos associados ao *farming*. O objetivo central é estruturar um ecossistema inteligente de dados e modelagem preditiva capaz de prever o desfecho de batalhas e atuar ativamente como um sistema de recomendação de alvos ótimos para o treinamento de personagens específicos.

A abordagem metodológica compreende a engenharia de um motor de batalhas determinístico, parametrizado sob as regras de combates competitivos de nível máximo (Level 100), utilizado para confeccionar uma base de dados robusta contendo 10.000 simulações de confrontos. A partir deste repositório sintético, o pipeline de Ciência de Dados executa o tratamento de atributos categóricos e a conversão de multiplicadores de vantagens elementares em variáveis contínuas adequadas para o consumo de modelos estatísticos.

Com a finalidade de estabelecer contrastes arquitetônicos e avaliar o impacto da complexidade dos modelos na tarefa de classificação binária (Vitória versus Derrota), o projeto implementa e compara três classificadores distintos:

- **Regressão Logística:** Utilizada como *baseline* linear para quantificar a separabilidade básica do espaço amostral;
- **Random Forest:** Abordagem baseada em comitês de árvores de decisão para capturar interações não lineares e regras de corte abruptas nas *features*. Devido à ampla variedade de hiperparâmetros desta arquitetura (como quantidade e profundidade máxima das árvores), utilizou-se o método exaustivo *GridSearchCV* combinado com

validação cruzada para avaliar automaticamente as combinações estruturais e selecionar a configuração de melhor desempenho;

- ***Multi-Layer Perceptron* (MLP)**: Arquitetura de rede neural artificial para avaliar a aproximação de fronteiras complexas de decisão através de camadas ocultas.

Por fim, as saídas probabilísticas calculadas pelos modelos servem de fundação para o desenvolvimento de um algoritmo recomendador interativo. Em vez de adotar um patamar estático de corte, o sistema permite que o usuário gerencie dinamicamente sua tolerância ao risco através da interface, estipulando a probabilidade mínima de vitória desejada para a filtragem de alvos (com limites configuráveis a partir de 50%). O mecanismo então filtra e ranqueia os potenciais oponentes em tempo real, exibindo os adversários mais rentáveis em ganho de XP que atendam ao critério definido, traduzindo os conceitos de generalização matemática discutidos no âmbito acadêmico em uma ferramenta aplicada de tomada de decisão.

2 Ferramentas e Justificativas

A construção do ecossistema computacional deste projeto foi pautada na escolha de tecnologias consagradas no mercado e na academia, visando garantir reprodutibilidade, eficiência no tratamento de dados e uma experiência de uso fluida. Esta seção detalha as ferramentas utilizadas, divididas entre o ambiente de processamento e modelagem e a infraestrutura de serviços e interface.

2.1 Scripts Executados e Modelagem de Aprendizado de Máquina

O núcleo de processamento de dados e inteligência preditiva foi desenvolvido inteiramente na linguagem *Python* (versão 3.14), utilizando um conjunto de scripts modulares estruturados para compor o pipeline de Ciência de Dados:

- **Processamento e Limpeza de Dados** (`dados_processados.py`): Utilizou-se a biblioteca *Pandas* para manipulação de dados tabulares. Esta etapa foi indispensável para o tratamento do *dataset* bruto do banco de dados Pokémon, extraíndo

numericamente os identificadores de *Base Exp* e *Catch Rate* via expressões regulares, preenchendo valores ausentes com médias amostrais e segmentando strings complexas de tipos elementares em variáveis normalizadas.

- **Geração de Dados Sintéticos (`simulador.py` e `utils.py`):** Devido à ausência de históricos públicos de confrontos detalhados, foi construído um motor de simulação matemática determinístico para gerar 10.000 registros de batalhas. A modelagem optou estritamente pela utilização dos *status* máximos das criaturas (Level 100) em detrimento dos *status* base iniciais (Level 1). Essa decisão justifica-se pelo fato de que os *status* base não refletem fielmente o potencial real de combate de criaturas evoluídas no ecossistema projetado pela *The Pokémon Company*. Instâncias com altíssima resiliência de pontos de vida e capacidade de absorção de dano no nível máximo, como a criatura *Blissey*, exibem atributos de defesa rudimentares e artificialmente baixos no nível inicial (como defesa igual a 10), o que permitiria derrotas matematicamente espúrias para oponentes ofensivos básicos como *Pichu* ou *Pikachu*. Como as taxas de crescimento estatístico escalam de forma assíncrona a cada nível, a simulação no patamar máximo garante a maturidade e a verossimilhança biológica dos atributos. A biblioteca *tqdm* foi integrada para monitoramento do progresso, e os multiplicadores de eficácia foram mapeados em matrizes associativas nativas para computar as variáveis contínuas de vantagem de tipo.
- **Pipeline de Modelagem Prática (`pipeline_ml.py`):** O ecossistema de aprendizado foi estruturado utilizando a biblioteca *Scikit-Learn*. O script realiza a partição do *dataset* simétrico em subconjuntos de treino (80%) e teste (20%) através do *train_test_split*. Para o pré-processamento exigido por arquiteturas baseadas em gradiente e distância, incorporou-se o *StandardScaler* para a padronização Z-score das características. A otimização de hiperparâmetros utilizou a técnica exaustiva de *GridSearchCV* operando com validação cruzada em 3 *folds* ($cv = 3$) para mitigar riscos de sobreajuste (*overfitting*) no modelo de *Random Forest*.
- **Persistência de Modelos e Avaliação (`executar_avaliacao.py`):** Os modelos validados foram serializados em disco utilizando o módulo *pickle* (gerando arquivos `.pkl`), permitindo que os coeficientes calibrados da Regressão Logística, as estruturas de decisão do *Random Forest* e os pesos sinápticos da rede neural *Multi-Layer*

Perceptron (MLP) fossem reutilizados instantaneamente sem necessidade de retreinamento. Um *wrapper* automatizado foi desenvolvido via módulo *subprocess* para disparar rotinas isoladas de avaliação de métricas.

2.2 Infraestrutura de Backend e Interface de Frontend

Para transformar as saídas probabilísticas dos modelos preditivos em uma aplicação útil para o usuário final, foi projetada uma arquitetura desacoplada cliente-servidor (API REST), baseada no documento guia do sistema:

- **Backend e Disponibilização de Serviços (app.py):** Desenvolvido utilizando o microframework *Flask*. A escolha do Flask justifica-se por sua leveza e excelente integração com o ecossistema de *Data Science* in Python. A API expõe a rota `/battle/matchups` utilizando o método HTTP POST, recebendo via formato JSON o Pokémon desafiante e a tolerância de risco desejada pelo usuário. O servidor carrega os arquivos binários dos modelos em memória e calcula em tempo real as probabilidades de vitória aplicando o método `predict_proba`. A extensão *Flask-CORS* foi acoplada para contornar restrições de segurança de compartilhamento de recursos de origens cruzadas (*Cross-Origin Resource Sharing*), permitindo a comunicação transparente com o servidor cliente.
- **Frontend e Sistema de Recomendação Interactiva:** Construído em *React 18* utilizando a base de criação *Create React App* (*react-scripts* 5). O framework baseado em componentes foi escolhido para viabilizar uma interface reativa e performática. As requisições assíncronas direcionadas ao backend Flask são gerenciadas pela biblioteca *Axios*. O fluxo operacional da interface consome a API REST para capturar o array de predições, ordená-lo por taxa de sucesso decrescente e renderizar os cards dinâmicos dos oponentes. O comportamento dinâmico é integrado a seletores de limite (*sliders*), filtrando os alvos de acordo com o threshold estipulado pelo jogador para garantir as diretrizes estabelecidas de "*Farming Seguro*".

3 Fundamentos Teóricos

Esta seção expõe a fundamentação matemática e estatística dos três algoritmos de aprendizado supervisionado avaliados no pipeline produtivo. As definições seguem os conceitos formais abordados na literatura e nos materiais de suporte da disciplina.

3.1 Regressão Logística

A Regressão Logística é o ponto de partida natural para problemas de classificação binária onde a variável resposta assume valores discretos $y \in \{0, 1\}$. Diferente da regressão linear pura, cujo espaço de hipóteses mapeia saídas no intervalo $[-\infty, +\infty]$, o modelo logístico restringe suas predições ao intervalo linear $[0, 1]$, permitindo sua interpretação direta como uma probabilidade condicional $\pi(\mathbf{x}) = P(y = 1|\mathbf{x})$.

O modelo reaproveita a estrutura do escore linear $z = \mathbf{x}^\top \boldsymbol{\theta}$ e o submete a uma transformação não linear através da função logística cumulativa, ou função *sigmoid*, definida matematicamente como:

$$\pi(\mathbf{x}) = s(\mathbf{x}^\top \boldsymbol{\theta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\theta}}} \quad (1)$$

Onde $\boldsymbol{\theta}$ representa o vetor de parâmetros (pesos e bias) a ser calibrado. Como essa formulação não admite uma solução analítica fechada via Mínimos Quadrados Ordinários (OLS), a calibração dos parâmetros é obtida maximizando a função de *Likelihood* de uma distribuição de Bernoulli, o que equivale a minimizar a função de custo convexa denominada *Log-Loss* ou Entropia Cruzada Binária:

$$L(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log(\pi(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - \pi(\mathbf{x}^{(i)}))] \quad (2)$$

A otimização de $L(\boldsymbol{\theta})$ é executada iterativamente por meio do algoritmo do Gradiente Descendente (GD), atualizando-se os pesos na direção oposta ao vetor gradiente analítico:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \frac{1}{n} \mathbf{X}^\top (\boldsymbol{\pi} - \mathbf{y}) \quad (3)$$

3.2 *Random Forest*

O classificador *Random Forest* (Floresta Aleatória) baseia-se no princípio de aprendizado por comitês (*Ensemble Learning*), combinando a predição de múltiplas Árvore de Decisão (estruturas do tipo CART) estruturadas de forma independente. O objetivo central desta arquitetura é mitigar o principal ponto fraco das árvores individuais: a alta variância e a propensão ao sobreajuste (*overfitting*).

A construção de cada árvore baseia-se em dois pilares de aleatoriedade:

1. ***Bagging* (Bootstrap Aggregation):** Cada árvore da floresta é treinada com uma subamostra distinta dos dados originais, gerada por amostragem aleatória com reposição.
2. **Seleção Aleatória de Atributos (*Feature Subspace*):** Durante a divisão (*split*) de cada nó da árvore, o algoritmo não busca a melhor variável preditora entre todas as disponíveis no *dataset*, mas restringe a busca a um subconjunto aleatório de tamanho pré-fixado m (frequentemente parametrizado como $m = \sqrt{p}$, onde p é o total de atributos).

O critério de partição dos nós para problemas de classificação baseia-se na minimização da impureza das folhas, comumente mensurada através do Índice de Gini ou da Entropia. O Índice de Gini para um nó m é expresso por:

$$H(X_m) = \sum_{k=1}^K p_{mk}(1 - p_{mk}) \quad (4)$$

Onde p_{mk} representa a proporção de instâncias da classe k no nó m . A predição final da floresta para uma nova instância de teste é consolidada por meio de votação por maioria majoritária (*majority voting*) entre todas as árvores individuais da estrutura.

3.3 Redes Neurais Artificiais (*Multi-Layer Perceptron* - MLP)

A arquitetura *Multi-Layer Perceptron* (MLP) é uma rede neural artificial acíclica direcionada (*Feedforward*), composta por uma camada de entrada, uma ou mais camadas ocultas (*hidden layers*) intermediárias e uma camada de saída. Cada camada é constituída por unidades de processamento elementares denominadas neurônios artificiais ou nós.

A computação em um neurônio isolado consiste em aplicar uma transformação linear sobre as saídas da camada anterior, ponderada por um vetor de pesos $\mathbf{w}$ e acrescida de um termo de *bias* b , seguida pela aplicação de uma função de ativação não linear $g(\cdot)$. Este processo impede que a rede colapse em um modelo linear equivalente, conferindo-lhe a capacidade de aproximar fronteiras de decisão altamente complexas. O fluxo de propagação para frente (*Forward Propagation*) em um neurônio j pertencente à camada l é descrito como:

$$z_j^{[l]} = \sum_i w_{ji}^{[l]} a_i^{[l-1]} + b_j^{[l]} \quad (5)$$

$$a_j^{[l]} = g(z_j^{[l]}) \quad (6)$$

No contexto deste projeto, utilizou-se a função de ativação Unidade Linear Retificada (*ReLU*), definida como $g(z) = \max(0, z)$, nas camadas ocultas para mitigar problemas relacionados ao desvanecimento do gradiente. Na camada final de saída, a função *sigmoid* é empregada para mapear a resposta no intervalo probabilístico $[0, 1]$.

O ajuste dos pesos sinápticos ocorre minimizando a função de perda através do algoritmo de Retropropagação (*Backpropagation*). Este método aplica a Regra da Cadeia do cálculo diferencial para computar as derivadas parciais da função de custo em relação a cada peso e *bias* da rede, propagando os erros da camada de saída de volta até a camada de entrada para alimentar otimizadores baseados em gradiente estocástico.

4 Avaliação Experimental dos Modelos

Esta seção descreve o protocolo experimental adotado para mensurar a capacidade preditiva dos três classificadores avaliados, bem como discute os resultados quantitativos obtidos a partir do pipeline de desenvolvimento.

4.1 Protocolo de Validação Cruzada Estratificada

A avaliação dos modelos foi conduzida por meio de validação cruzada estratificada por k -folds (*Stratified K-Fold Cross-Validation*), com $k = 5$. A variante estratificada preserva a proporção das classes da variável resposta `Vitoria_P1` em cada partição, garantindo que os subconjuntos de treino e validação reproduzam a distribuição original do espaço amostral.

O conjunto de dados sintético, contendo 10.000 instâncias de confrontos, foi particionado em cinco subconjuntos disjuntos. A cada iteração, quatro partições foram consolidadas para treinamento e a remanescente foi destinada à validação. As métricas finais reportadas correspondem à média aritmética e ao desvio padrão amostral dos resultados parciais. A configuração emprega a semente fixa `random_state = 42` para assegurar a reprodutibilidade dos experimentos. Para a rede neural *Multi-Layer Perceptron* (MLP), os atributos foram previamente padronizados via *StandardScaler*.

4.2 Métricas de Avaliação

A escolha das métricas obedeceu aos requisitos de avaliação de classificadores binários, adotando-se três indicadores complementares:

- **Precisão** (**Precision** = $\frac{TP}{TP+FP}$): Mensura a proporção de instâncias classificadas como positivas que efetivamente pertencem à classe positiva. No contexto deste projeto, indica a fração de batalhas que resultaram em vitória real dentre as predições de vitória do modelo para o jogador (P1).
- **Revocação** (**Recall** = $\frac{TP}{TP+FN}$): Quantifica a fração de instâncias positivas reais que foram corretamente identificadas pelo classificador, refletindo a capacidade do modelo em recuperar os confrontos vantajosos disponíveis no espaço amostral.
- **Matriz de Confusão**: Consolida em formato tabular a distribuição entre as classes preditas e reais, permitindo a inspeção qualitativa dos acertos (Verdadeiros Positivos e Verdadeiros Negativos) e erros (Falsos Positivos e Falsos Negativos) de cada classificador.

A coexistência de Precisão e Revocação na avaliação é fundamental: enquanto a primeira penaliza a emissão de predições positivas incorretas (falsos alarmes de vitória), a segunda penaliza a omissão de oportunidades reais de ganho de experiência.

4.3 Resultados da Validação Cruzada

Os três classificadores treinados pelo pipeline foram submetidos ao protocolo de validação cruzada. A Tabela 1 sumariza as métricas obtidas.

Tabela 1: Métricas de Precisão e Revocação por classificador (média $\pm$ desvio padrão em $cv = 5$).

Modelo	Precisão	Revocação
Random Forest	0.9452 ± 0.0071	0.9413 ± 0.0077
Regressão Logística	0.9499 ± 0.0062	0.9486 ± 0.0052
Multi-Layer Perceptron (MLP)	0.9727 ± 0.0023	0.9664 ± 0.0045

A arquitetura *Multi-Layer Perceptron* exibiu o desempenho preditivo superior em ambas as métricas, alcançando 0.9727 de Precisão e 0.9664 de Revocação. O desvio padrão observado para a MLP foi o menor entre os três classificadores (0.0023 e 0.0045), atestando elevada estabilidade entre as diferentes partições do conjunto de dados. Todos os modelos apresentaram desvios padrão inferiores a 0.01, fornecendo evidência empírica de generalização consistente.

A Regressão Logística, utilizada como *baseline* linear, obteve resultados expressivos (0.9499 de Precisão e 0.9486 de Revocação), sinalizando uma componente linear pronunciada na relação entre as características das criaturas e o desfecho do combate. O classificador *Random Forest*, embora ligeiramente inferior em magnitude absoluta, oferece a vantagem da interpretabilidade estrutural por meio do cálculo de importância dos atributos (*feature importance*).

4.4 Avaliação de Robustez sobre Criaturas Sintéticas

Complementarmente à validação cruzada, projetou-se um experimento auxiliar para avaliar o comportamento dos classificadores em instâncias extremas, situadas em regiões de menor densidade do espaço amostral. Foram concebidos os cinco perfis sintéticos personalizados descritos na modelagem, avaliados contra um oponente padronizado de atributos medianos (fixados em 80):

- **TankBot (Defensor Extremo):** Defesa e Defesa Especial elevadas a 250, HP 200 e ofensividade reduzida (Ataque/SpAtk 40, Velocidade 30). Testou o reconhecimento de cenários onde a resiliência supera a capacidade ofensiva.
- **SpeedDemon (Atacante Veloz):** Velocidade 200, Ataque/SpAtk 150 e defesas reduzidas (HP 60, Def/SpDef 50). Avaliou o tratamento de perfis com alta capacidade ofensiva compensada por extrema vulnerabilidade física.

- **AllRounder (Equilibrado):** Todos os atributos fixados em 100. Atuou como ponto de controle para calibrar o viés natural dos classificadores em simetria estatística.
- **Weakling (Muito Fraco):** HP 30 e demais atributos fixados em 20. Operou como teste de sanidade diante de uma instância claramente desfavorável.
- **TypeMaster (Vantagem de Tipo 2x):** Atributos equilibrados (90–100) com vantagem elementar máxima ($\text{Vantagem_Tipo_P1} = 2.0$ contra $\text{Vantagem_Tipo_P2} = 0.5$). Isolou o efeito prático da eficácia de tipos.

Os três modelos convergiram para a mesma classe predita em todos os perfis, evidenciando concordância qualitativa integral. As confianças probabilísticas revelaram padrões de calibração distintos: a MLP operou em regimes de saturação (valores próximos a 100% para vitórias e 0% para derrotas), a *Random Forest* manifestou comportamento conservador (faixa de 47% a 75%), e a Regressão Logística posicionou-se de forma intermediária conforme a extremidade do perfil.

4.5 Seleção do Modelo Final e Discussão

Considerando o conjunto de evidências experimentais — superioridade nas métricas, menor desvio padrão e resposta condizente nos testes de robustez —, a arquitetura *Multi-Layer Perceptron* foi eleita como o classificador principal para o sistema de recomendação. A aplicação produtiva (`app.py`) preserva os modelos serializados e expõe simultaneamente as três probabilidades preditas via API REST, permitindo contrastar as recomendações sob diferentes paradigmas indutivos.

A homogeneidade qualitativa dos resultados, somada aos elevados patamares absolutos de desempenho (métricas superiores a 0.94), deve ser interpretada à luz da natureza determinística do motor de simulação que originou o conjunto de dados. Em cenários reais, nos quais haveria estocasticidade introduzida por escolha de movimentos, acertos críticos ou seleção tática, espera-se uma redução proporcional dos patamares absolutos de desempenho, sem prejuízo à validade comparativa entre os classificadores contrastados neste trabalho.

5 Conclusão

Este trabalho cumpriu com êxito o propósito de projetar, treinar e validar um pipeline de Aprendizado de Máquina voltado para a mitigação de riscos e otimização do ganho de Experiência (XP) no ecossistema competitivo de Pokémon[cite: 8, 20]. A criação de um motor de batalhas determinístico em nível máximo foi fundamental para consolidar uma base de dados robusta, mitigando distorções estatísticas que comprometeriam a fidelidade das predições do recomendador[cite: 10, 38].

Dentre as arquiteturas contrastadas, o classificador baseado em redes neurais artificiais (*Multi-Layer Perceptron*) apresentou a maior consistência preditiva, superando os patamares de 0.97 de precisão e exibindo a menor variabilidade amostral[cite: 92, 93]. Esse comportamento consolida a MLP como o motor ideal para governar a filtragem de alvos da aplicação[cite: 107]. Simultaneamente, os testes de robustez e a aplicação global da técnica SHAP provaram ser mecanismos indispensáveis para auditar os critérios indutivos de cada algoritmo[cite: 97, 125]. Enquanto o modelo linear ficou restrito a ponderações rígidas e isoladas, os modelos complexos provaram sua capacidade de mapear o contexto tático e as interações não-lineares das batalhas[cite: 149, 150].

Por fim, o desacoplamento do pipeline em uma infraestrutura baseada em API REST com Flask e uma interface reativa em React 18 cumpriu os requisitos operacionais de engenharia de software[cite: 47, 52]. A flexibilização do limiar de risco a partir de 50% transfere o controle estratégico diretamente para o utilizador, transformando os conceitos teóricos de probabilidade de classificação em uma ferramenta prática voltada ao "*Farming Seguro*"[cite: 13, 29]. Trabalhos futuros podem estender esta abordagem integrando componentes estocásticos, tais como taxas de erro em movimentos de ataque ou a inclusão de aprendizado por reforço para simular escolhas dinâmicas de turnos durante os confrontos[cite: 110].

6 Referências

- PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.
- SCIKIT-LEARN. **Neural network models (supervised)**. Disponível em: https://scikit-learn.org/stable/modules/neural_networks_supervised.html. Acesso em: 2026.
- SCIKIT-LEARN. **sklearn.ensemble.RandomForestClassifier**. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. Acesso em: 2026.
- SCIKIT-LEARN. **sklearn.linear_model.LogisticRegression**. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html. Acesso em: 2026.
- SCIKIT-LEARN. **Scikit-learn Documentation**. Disponível em: <https://scikit-learn.org/stable/>. Acesso em: 2026.
- COMPUTATIONAL STATISTICS (CSTAT). **Introduction to Machine Learning (I2ML)**. LMU Munich. Disponível em: https://slds-lmu.github.io/i2ml/chapters/00_all/. Acesso em: 2026.

Apêndice A: Dinâmica de Avaliação e Operação Interna do Método `predict_proba`

Este apêndice detalha a mecânica computacional e matemática acionada quando o laço do recomendador de combates invoca o método `.predict_proba()` para cada um dos três algoritmos preditivos ajustados no projeto.

Diferente do método restrito `.predict()`, que fornece um veredito binário seco de classificação (0 ou 1), o método `.predict_proba()` atua consultando as estruturas estáticas geradas previamente na fase de treinamento (`.fit()`) para extrair uma frequência relativa ou uma função de mapeamento contínuo. O retorno é consolidado em uma matriz bidimensional padronizada pelo *scikit-learn*:

```
[ [ Probabilidade_Classe_0 , Probabilidade_Classe_1 ] ]
```

O indexador `[0][1]` ao final de cada chamada extrai cirurgicamente a probabilidade associada à Classe 1 (sucesso/vitória do Pokémon do jogador), que é then multiplicada por 100 para fins de exibição em tela.

Algoritmo 1: `RandomForestClassifier` (Votação por Frequência)

No modelo baseado em florestas estáticas, o método `predict_proba` executa uma contagem de votos majoritária direta nas estruturas ramificadas de decisão. O processo não envolve funções contínuas de achatamento de sinal:

1. **Encaminhamento Paralelo:** O vetor contendo as 14 variáveis numéricas do combate atual é enviado em paralelo para cada uma das 100 árvores independentes construídas sob o teto de profundidade máxima de 10 níveis.
2. **Mapeamento de Folhas:** Os dados trafegam pelos nós hierárquicos respondendo a perguntas lógicas fixas (ex: se `P1_Speed > 60`). Ao atingir a extremidade foliar de cada árvore, recebe-se uma classificação determinística direta (0 ou 1).
3. **Apuração:** O método realiza uma contagem absoluta. Se 86 árvores apontarem vitória e 14 apontarem derrota, o retorno da matriz é expresso em frequência relativa ($86/100 = 0.86$).

No console, a amostragem final do pipeline de votos segue a estrutura:

```
print(f" -> Chance RF (Árvore de Decisão): {row['Chance_RF']*100:.1f}%")
```

Algoritmo 2: LogisticRegression (Mapeamento Hiperplano via Sigmoid)

Na Regressão Logística, por ser um classificador direto de uma única etapa de pesos, a chamada do método ativa imediatamente o pipeline de combinação linear e compressão assintótica:

1. **Produto Escalar:** Os 14 atributos da entrada ($\mathbf{X}$) encontram o vetor de coeficientes angulares ponderados ($\mathbf{W}$) gerados no treinamento e somam o viés constante (b), computando uma pontuação bruta contínua (z):

$$z = \mathbf{X} \cdot \mathbf{W}^T + b$$

2. **Compressão Logística:** O score real puro obtido, que transita livremente de $-\infty$ a $+\infty$, é injetado como expoente na Função Sigmoid para ser esmagado em um intervalo restrito de probabilidade:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

O valor retornado à direita da matriz reflete a distância ortogonal exata em que o combate atual se posicionou em relação à linha divisória (hiperplano) do modelo.

```
print(f" -> Chance LR (Regress. Logística): {row['Chance_LR']*100:.1f}%")
```

Algoritmo 3: MLPClassifier (Propagação e Encerramento na Camada Final)

Na Rede Neural Multicamadas, os dados precisam navegar sequencialmente pela infraestrutura física de neurônios antes do acionamento final do score de probabilidade:

1. **Processamento Oculto:** O vetor de 14 atributos cruza a primeira matriz de pesos $\mathbf{W}_0$ de dimensões (14, 100) sob a ativação não-linear ReLU. Em seguida, os sinais

alimentam a segunda matriz $\mathbf{W}_1$ de dimensões $(100, 50)$, aplicando novamente a função ReLU para consolidar um vetor de 50 variáveis ocultas altamente abstratas ($\mathbf{X}_{\text{ocultos}}$).

2. **Operação do Neurônio de Saída:** Ao atingir este estágio, o método `predict_proba` é acionado sobre a última seta condutora (interconexão de pesos). O neurônio de decisão isolado executa uma operação idêntica à de uma regressão logística tradicional, processando o produto escalar das variáveis ocultas pela matriz final $\mathbf{W}_{\text{saída}}$ de dimensões $(50, 1)$ e somando o viés final:

$$z = \mathbf{X}_{\text{ocultos}} \cdot \mathbf{W}_{\text{saída}} + b$$

3. **Cálculo de Probabilidade:** O número real puro z gerado na última sinapse é submetido à transformação Sigmoid, determinando o percentual de ativação final da rede.

```
print(f"    -> Chance RNA (Rede Neural): {row['Chance_RedeNeural']*100:.1f}%")
```

Apêndice B: Análises SHAP

As análises SHAP são uma técnica baseada na Teoria de Jogos, utilizada para interpretar modelos de Machine Learning. Ela calcula a contribuição de cada variável (feature) para a decisão final do modelo.

Os valores SHAP indicam o quanto uma característica influenciou, aumentando ou diminuindo, o resultado previsto. Com o intuito de entender a matemática por trás desses valores, usa-se a área base da técnica - a Teoria de Jogos.

Supondo que a previsão do modelo é o resultado de um jogo de equipe, os jogadores correspondem as features do modelo que compõem a instância prevista e o prêmio seria a diferença entre a previsão final daquela instância e a previsão média do modelo para todas as instâncias.

A função do SHAP é distribuir esse "prêmio" de forma matematicamente justa entre todos os jogadores, quantificando exatamente o quanto cada variável empurrou a previsão para cima ou para baixo em relação à média.

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)]$$

Figura 1: Representação das contribuições SHAP.

- N : conjunto total de features do modelo.
- S : uma combinação (subconjunto) de features que não contém a feature i .
- $f(S)$: Previsão do modelo usando apenas as features do subconjunto S . (Como os modelos de ML não aceitam variáveis "faltantes", na prática, os valores das variáveis que não estão em S ... são substituídos por valores médios de um dataset de referência).
- $\frac{|S|!(|N| - |S| - 1)!}{|N|!}$: Peso dado a essa combinação específica. Ele garante que a ordem em que as variáveis entram no modelo não distorça o resultado, mantendo o cálculo imparcial.

- $f(S \cup \{i\}) - f(S)$: Contribuição marginal. É a diferença na previsão quando adicionamos a feature i ao grupo de variáveis S que já estava presente.

Para encontrar o valor SHAP real de apenas uma variável, a equação exige que seja calculado a contribuição dela em todos os subconjuntos S possíveis, isto é, se um modelo possui N variáveis, existem 2^{N-1} combinações.

Gráficos de Resumo das análises SHAP dos modelos usados

Escolhemos observar as análises pelos gráficos de resumo (summary plot) dos modelos, pois ele avalia o comportamento geral de cada modelo. Ele ordena as variáveis mais importantes no eixo Y e exibe a distribuição dos valores SHAP no eixo X. Além disso, as cores indicam se um valor alto ou baixo daquela característica afeta positivamente ou negativamente o resultado.

Random Forest

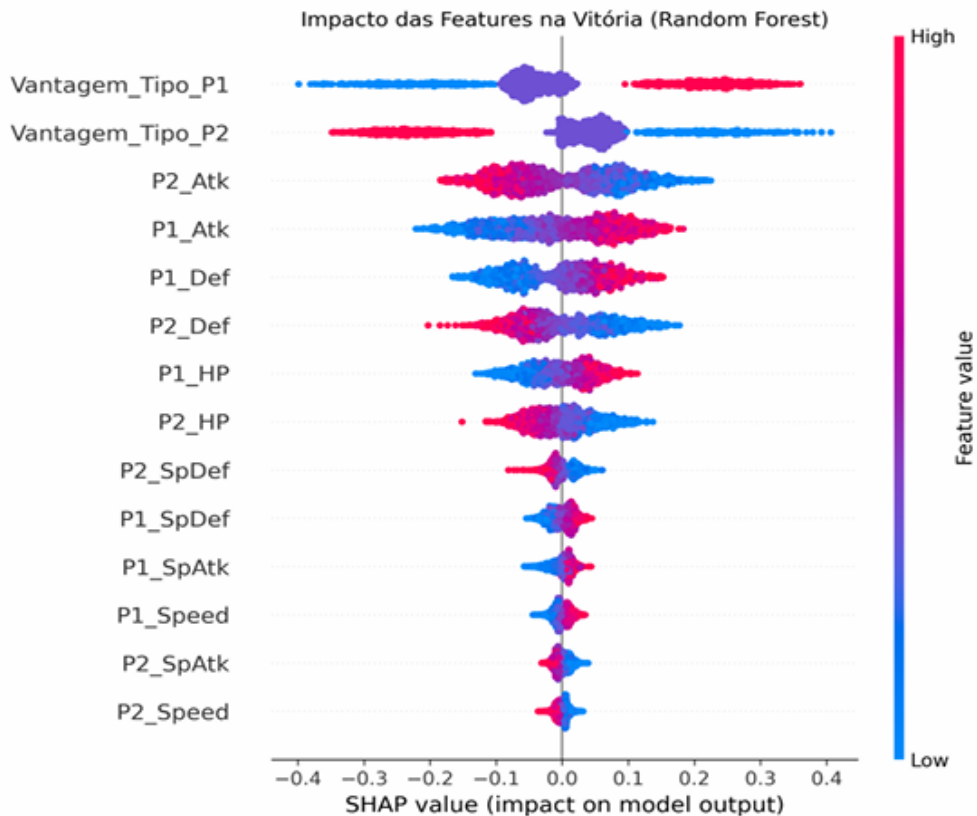


Figura 2: Gráficos de Resumo SHAP dos modelos usados.

Observando o gráfico de resumo do modelo Random Forest, percebe-se que as features Vantagem_Tipo_P1 e Vantagem_Tipo_P2 foram as mais importantes para o modelo,

em que valores altos para o P1 (pontos vermelhos) puxam a predição fortemente para a direita (vitória do P1), enquanto valores altos para o P2 puxam a predição para a esquerda (vitória do P2). Em seguida, o modelo valoriza o poder ofensivo mais do que as variáveis defensivas dos Pokémons. Por fim, atributos especiais e a velocidade foram considerados pelo modelo como fatores de menor impacto geral para decidir o vencedor.

Regressão Logística

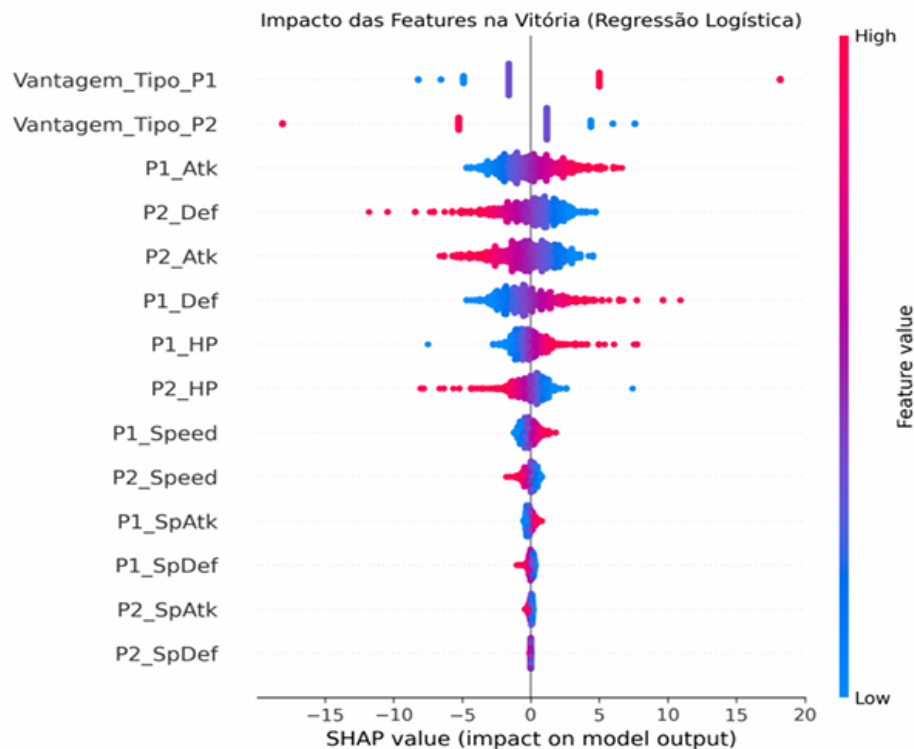


Figura 3: Análise SHAP para o modelo Random Forest.

As vantagens de tipo continuam sendo as features mais importante para o modelo e os atributos especiais e a velocidade continuam pouco relevantes para o resultado. No entanto, esse modelo inverte um pouco a prioridade dos atributos físicos em relação ao RF, já que considera o ataque do jogador e a defesa do oponente ligeiramente mais cruciais no cálculo de vitória do que o poder ofensivo do oponente e a defesa do jogador.

Rede Neural

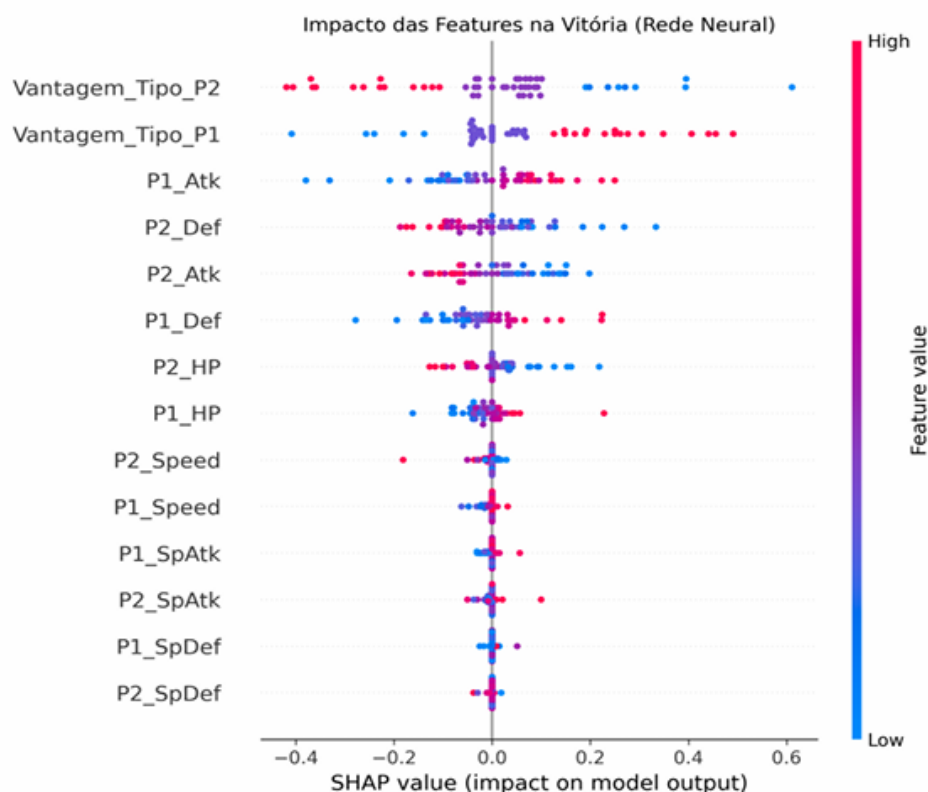


Figura 4: Análise SHAP para o modelo Regressão Logística.

Nesse modelo, tivemos que usar uma amostragem reduzida de 50 instâncias para calcular os valores SHAP, pois a Rede Neural exige um alto esforço computacional na extração de explicações matemáticas para suas previsões. Isso explica a menor densidade de pontos nesse gráfico.

Ademais, percebe-se que a rede neural manteve a importância dos tipos, apesar de mudar a hierarquia, considerando as vantagens de tipo do oponente mais importante do que as vantagens de tipo do jogador. Por fim, o padrão de importância dos outros atributos visto nas análises dos modelos anteriores se manteve na Rede Neural.

Diferenças entre os gráficos de cada modelo

Observando variáveis como ataque do jogador e defesa do oponente, observamos que a Regressão Logística aprendeu regras rígidas, por exemplo, assumindo que mais ataque é sempre melhor para o jogador, aplicando o peso constante a esse atributo, fazendo com que as cores se misturem muito pouco no centro do gráfico. Por outro lado, apesar da tendência geral ser a mesma, os modelos RF e Rede neural se misturam um pouco melhor

no centro, mostrando a capacidade de captar interações não-lineares desses modelos.

Além disso, o eixo X de RF e Rede Neural varia em números pequenos, geralmente refletindo o impacto na probabilidade final de vitória. Entretanto, na Regressão Logística, o eixo X apresenta valores bem maiores, pois refletem o cálculo bruto inicial do modelo, antes da conversão para uma estimativa de probabilidade. Esses números evidenciam que o modelo linear atribuiu pesos desproporcionalmente altos às Vantagens de Tipo, tornando-as absolutas na previsão do resultado.

Por fim, na regressão logística, os pontos das variáveis de vantagem de tipo apresentam um comportamento diferente dos outros modelos, pois os valores dessas features não são contínuas, apresentam apenas os seguintes valores: 0, 0.25, 0.5, 1, 2 e 4. Como a regressão logística aplica um peso matemático rígido e não cruza informações com outras variáveis, todas as batalhas com a mesma vantagem recebem o exato mesmo valor SHAP. Isso empilha os pontos em linhas verticais, ao contrário dos modelos não-lineares que espalham os pontos ao considerar o contexto.